

Data Structures & Algorithms

Laboratory – Algorithms, Searching & Complexity in Java

Dr. Yatish Bathla, PhD. (Information Technology), Obuda University, Budapest

Lab objectives and setup

Write simple array algorithms in Java

Compare linear and binary search

Count operations and time the code

Classify code by Big-O

Setup: JDK 17+, any IDE, one .java file per algorithm

JDK 17 Installation Link: https://www.youtube.com/watch?v=cL4GcZ6GJV8

IntelliJ Community Edition Installation Link:
https://www.youtube.com/watch?v=MBbRyXIQ89Q

Starter code: one class per algorithm

```
public class MinMaxAverage {
    public static void main(String[] args) {
        int[] a = {4, 9, 2, 7};
        // call your method here
        // print the result
    }
}
```

Setup tips

File name = class name

One class = one algorithm

Test and print in main

Test every method with these cases

Case	Example	Why it matters
Typical	{4, 9, 2, 7}	Normal behaviour
Single item	{5}	Loop boundaries
Empty	{ }	Crash or exception?
Duplicates	{3, 3, 3}	Which index is returned?
Key absent	search for 99	Return value -1

Lab plan: 60 minutes

10

Warm-up

Ex 1-2: max, min, sum, reverse.

15

Searching

Ex 3-4: linear and binary search.

15

Complexity

Ex 5-6: shifting, Big-O table.

15

Measure

Ex 7: timing experiment.

Remaining 5 minutes: share results and submit.

Exercise 1: MinMaxAverage

```
int min = a[0], max = a[0];
long sum = 0;
for (int v : a) {
    if (v < min) min = v;
    if (v > max) max = v;
    sum += v;
}
double avg = (double) sum / a.length;
```

Tasks

Test: {4, 9, 2, 7} gives min 2, max 9, avg 5.5

How many comparisons for n items?

What is the time complexity?

Exercise 2: ReverseArray

```
int i = 0, j = a.length - 1;
while (i < j) {
    int tmp = a[i];
    a[i] = a[j];
    a[j] = tmp;
    i++;
    j--;
}
```

Tasks

Two pointers move toward the middle

Test: {1, 2, 3, 4, 5} becomes {5, 4, 3, 2, 1}

How many swaps? What complexity?

Exercise 2b: SecondLargest

```
Integer first = null, second = null;
for (int v : a) {
    if (first == null || v > first) {
        second = first;
        first = v;
    } else if (v != first &&
        (second == null || v > second)) {
        second = v;
    }
}
```

Tasks

One pass, no sorting

Test: {4, 9, 2, 7} gives 7

Test: {5, 5, 5} gives null (no second value)

Exercise 3: LinearSearch

```
static long comparisons = 0;

static int linearSearch(int[] a, int key) {
    for (int i = 0; i < a.length; i++) {
        comparisons++;
        if (a[i] == key) return i;
    }
    return -1;
}
```

Tasks

Returns the index, or -1 if absent

Counter records comparisons

Test best case (a[0]) and worst case (absent)

Exercise 4: BinarySearch

```
static int binarySearch(int[] a, int key) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        comparisons++;
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Tasks

Array must be sorted

Each step halves the range

Test on a sorted array of 16 items

Exercise 4b: BinarySearchRecursive

```
static int search(int[] a, int key,
                  int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] < key)
        return search(a, key, mid + 1, hi);
    return search(a, key, lo, mid - 1);
}
```

Tasks

Same logic as the loop version

Base case: empty range

Call: search(a, key, 0, a.length - 1)

Record your comparison counts

n	Linear, worst case	Binary, worst case (expected)
1,000	1,000	10
10,000	10,000	14
100,000	100,000	17
1,000,000	1,000,000	20

Run ComparisonCounts.java, then plot comparisons against n.

Exercise 5: DeleteFromArray

```
// delete element at index pos
int shifts = 0;
for (int i = pos; i < size - 1; i++) {
    a[i] = a[i + 1];
    shifts++;
}
size--;
```

Tasks

Shift left to close the gap

Delete a[0] in an array of 8: 7 shifts

Delete the last item: 0 shifts

Exercise 5b: InsertIntoSortedArray

```
// a is sorted, size < a.length
int i = size - 1;
while (i >= 0 && a[i] > x) {
    a[i + 1] = a[i];
    i--;
}
a[i + 1] = x;
size++;
```

Tasks

Shift larger items right, then place x

Insert 10 into {5, 8, 12, 20}

Best and worst case?

Exercise 6: BigOExamples

Snippet	Your answer
for (i = 0; i < n; i += 2) work();	
for i < n: for j < n: work();	
for (i = 1; i < n; i *= 2) work();	
for i < n: for j < i: work();	
loop to n, then a second loop to n	

Exercise 7: TimingExperiment

```
long t0 = System.nanoTime();
int r = linearSearch(a, key);
long t1 = System.nanoTime();
System.out.println((t1 - t0) + " ns");
```

Experiment

Run each search 5 times, take the median

First run is slower (JVM warm-up)

Try n = 1e5, 1e6, 1e7

Debugging checklist

Read the exception message and line number first

ArrayIndexOutOfBounds: check loop bounds

Infinite loop: check how lo and hi change

Wrong result: print variables inside the loop

Shrink to the smallest failing input

Optional challenges

MissingNumber: use the sum formula

RemoveDuplicatesSorted: write pointer

RotateArray: reverse three times

TwoSumSorted: two pointers

State time and extra space for each

Reflection questions for your report

Did your counts match n and log n?

Why do timings vary between runs?

Which exercise was the slowest, and why?

When is linear search better than binary search?

Why are arrays costly for frequent insertion?

Report template

Section	What to include
1. Code	One .java file per algorithm, with comments
2. Tests	Inputs and outputs for every method
3. Data	Comparison counts and timings
4. Analysis	Big-O answers with one-line reasons
5. Reflection	Answers to the five questions